KING MONGKUT'S UNIVERSITY OF TECHNOLOGY NORTH BANGKOK
Faculty of Applied Science / Faculty of Information Technology
FINAL EXAMINATION 2026 — MASTER SOLUTIONS & GRADING KEY (DETAILED V3)

Course: 060243106 & 060223119 Data Structures and Algorithms | Instructor: Assoc. Prof. Dr. Pradit Pitaksatheinkul
Total Score: 70 Raw Marks (35% Final Grade) | Complete Mathematical Verification & Rubric

Question 1: Hashing & Open Addressing with Double Hashing (10 Marks) — Master Solutions

Formulas: TableSize = 19 | $h_0(X) = X \bmod 19$ | $hash_2(X) = 17 - (X \bmod 17)$ | $h_i(X) = (h_0(X) + i \times hash_2(X)) \bmod 19$

1.1 Step-by-Step Probe Calculation & Collision Tracing: [3 Marks]

Key (X)	$h_0(X) = X \bmod 19$	$hash_2(X) = 17 - (X \bmod 17)$	Probe Sequence & Collision Trace	Final Slot	Collisions
26	26 mod 19 = 7	17 - (26 mod 17) = 8	Probe 0: Slot 7 is EMPTY → Placed	**7**	**0**
45	45 mod 19 = 7	17 - (45 mod 17) = 6	Probe 0: Slot 7 (Collides with 26) → Probe 1: (7+6)%19 = 13 (Empty)	**13**	**1**
70	70 mod 19 = 13	17 - (70 mod 17) = 15	Probe 0: Slot 13 (Collides with 45) → Probe 1: (13+15)%19 = 28%19 = 9 (Empty)	**9**	**1**
85	85 mod 19 = 9	17 - (85 mod 17) = 17	Probe 0: Slot 9 (Collides 70) → Probe 1: (9+17)%19 = 7 (Collides 26) → Probe 2: (9+34)%19 = 5 (Empty)	**5**	**2**
102	102 mod 19 = 7	17 - (102 mod 17) = 17	Probe 0: Slot 7 (Collides 26) → Probe 1: (7+17)%19 = 5 (Collides 85) → Probe 2: (7+34)%19 = 3 (Empty)	**3**	**2**
123	123 mod 19 = 9	17 - (123 mod 17) = 13	Probe 0: Slot 9 (Collides 70) → Probe 1: (9+13)%19 = 3 (Collides 102) → Probe 2: (9+26)%19 = 16 (Empty)	**16**	**2**
143	143 mod 19 = 10	17 - (143 mod 17) = 10	Probe 0: Slot 10 is EMPTY → Placed	**10**	**0**
155	155 mod 19 = 3	17 - (155 mod 17) = 15	Probe 0: Slot 3 (Collides with 102) → Probe 1: (3+15)%19 = 18 (Empty)	**18**	**1**

1.2 Final Hash Table Array Representation (Element & Info): [2 Marks]

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Element	-	-	-	**102**	-	**85**	-	**26**	-	**70**	**143**	-	-	**45**	-	-	**123**	-	**155**
Info	-	-	-	**A**	-	**A**	-	**A**	-	**A**	**A**	-	-	**A**	-	-	**A**	-	**A**

1.3 Collisions Breakdown & Max Colliding Keys: [2 Marks]

- Keys with 0 collisions: 26, 143 (2 keys) | Keys with 1 collision: 45, 70, 155 (3 keys)
- Keys with MAXIMUM collisions: 85, 102, 123 (Each suffered 2 collisions)
  → Key 85 collided at slots: 9, 7 | Key 102 collided at slots: 7, 5 | Key 123 collided at slots: 9, 3
- Total cumulative collisions: 0 + 1 + 1 + 2 + 2 + 2 + 0 + 1 = 9 collisions (Count each collision step as 1).

1.4 Load Factor & Rehashing Threshold Analysis: [3 Marks]

- Current Load Factor (λ): λ = N / TableSize = 8 / 19 = 0.4211 (42.11%)
- Rehashing Threshold at λ ≥ 80%:
  (a) Maximum elements before Rehashing: ☐19 × 0.8☐ = 16 elements (or ☐19 × 0.8☐ = 15).
  (b) Keys to insert before Rehashing: 16 - 8 = 8 more keys (or 15 - 8 = 7 more keys).
  (c) New Table Size: Must be the smallest prime number ≥ 2 × Old TableSize = 2 × 19 = 38 → Next prime is 41 (or 43, 47).

Question 2: Priority Queue (Binary Min-Heap) — Master Solutions & Tracing (10 Marks)

Input Sequence: [64, 14, 24, 1, 20, 2, 5, 10, 6, 8, 15, 20] (12 keys).

2.1 Fundamental Properties & 1D Array Formulas: [1 Mark]

- Structure Property: A complete binary tree, filled completely at all levels except possibly the lowest, which is filled from left to right.
- Heap-Order Property: For every node X other than the root, key(Parent(X)) ≤ key(X). (Root contains minimum element).
- Formulas for Index i: Left Child = 2i | Right Child = 2i + 1 | Parent = □i / 2□ (i // 2)
  Why index 0 is unused: Allows direct multiplication/division by 2 without offset calculations.

2.2 Four Vertical Array Tracing Tables (Final States): [4 Marks]

Index	2.1 After 12 Inserts	2.2 After deleteMin() #1	2.3 After Insert 3	2.4 After deleteMin() #2
[0] (Unused)	-	-	-	-
[1]	1	2	2	3
[2]	6	6	6	6
[3]	2	5	3	5
[4]	10	10	10	10
[5]	8	8	8	8
[6]	5	20	5	20
[7]	24	24	24	24
[8]	64	64	64	64
[9]	14	14	14	14
[10]	20	20	20	20
[11]	15	15	15	15
[12]	20	-	20	-

2.3 Tree Representations & Level-by-Level Verification: [2 Marks]

- Tree 2.1 (After 12 Inserts): Root = 1
  Level 1: Left = 6, Right = 2
  Level 2: Children of 6 are (10, 8); Children of 2 are (5, 24)
  Level 3: Children of 10 are (64, 14); Children of 8 are (20, 15); Child of 5 is (20)
- Tree 2.4 (Final State after deleteMin #2): Root = 3
  Level 1: Left = 6, Right = 5
  Level 2: Children of 6 are (10, 8); Children of 5 are (20, 24)
  Level 3: Children of 10 are (64, 14); Children of 8 are (20, 15)

2.4 'hole' Variable Tracing Walkthrough: [2 Marks]

- deleteMin() #1 Walkthrough:
  1. Extract min element: arr[1] = 1 is returned. CurrentSize decrements to 11.
  2. Last element temp = arr[12] = 20 is saved. Hole starts at root: hole = 1.
  3. Children of hole 1 are arr[2] = 6 and arr[3] = 2. Smaller child is 2 at index 3.
  4. Since temp (20) > 2, shift 2 up to arr[1]. Hole moves to hole = 3.
  5. Children of hole 3 are arr[6] = 5 and arr[7] = 24. Smaller child is 5 at index 6.
  6. Since temp (20) > 5, shift 5 up to arr[3]. Hole moves to hole = 6.
  7. Left child of hole 6 is 2 × 6 = 12 > CurrentSize (11). No children exist. Loop terminates.
  8. Place temp = 20 into arr[6]. Complete!
- Insert Key 3 Walkthrough:
  1. CurrentSize increments to 12. Hole starts at index hole = 12, temp = 3.
  2. Parent is at □12/2□ = 6 (arr[6] = 20). Since 3 < 20, shift 20 down to arr[12]. hole = 6.
  3. Parent is at □6/2□ = 3 (arr[3] = 5). Since 3 < 5, shift 5 down to arr[6]. hole = 3.
  4. Parent is at □3/2□ = 1 (arr[1] = 2). Since 3 > 2, loop terminates. Place 3 at arr[3]!

2.5 Time Complexity Analysis: [1 Mark]

- Insert: Worst Case = O(log N) (element percolates to root), Average Case = O(1) (moves up ~1.607 levels).
- deleteMin: Worst Case = O(log N) (element percolates down to bottom leaf level).

Question 3: Insertion Sort Tracing & Position Moves — Master Solutions (10 Marks)

3.1 Worst-Case Analysis (N = 12 Elements): [3 Marks]

- Initial Worst-Case Array: [12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1] (Strictly descending order).
- Formula for Total Position Moves: Σ_{p=1}^{N-1} p = N(N - 1) / 2.
- Calculation for N = 12: 12 × 11 / 2 = 66 position moves (shifts) | Comparisons = 66 comparisons.

3.2 Array Tracing Table: Given input [61, 52, 12, 24, 8, 35, 32, 5, 71, 68, 18, 2]: [4 Marks]

Pass (p)	Key	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	Moves
Initial	-	61	52	12	24	8	35	32	5	71	68	18	2	-
p = 1	52	52	61	12	24	8	35	32	5	71	68	18	2	**1**
p = 2	12	12	52	61	24	8	35	32	5	71	68	18	2	**2**
p = 3	24	12	24	52	61	8	35	32	5	71	68	18	2	**2**
p = 4	8	8	12	24	52	61	35	32	5	71	68	18	2	**4**
p = 5	35	8	12	24	35	52	61	32	5	71	68	18	2	**2**
p = 6	32	8	12	24	32	35	52	61	5	71	68	18	2	**3**
p = 7	5	5	8	12	24	32	35	52	61	71	68	18	2	**7**
p = 8	71	5	8	12	24	32	35	52	61	71	68	18	2	**0**
p = 9	68	5	8	12	24	32	35	52	61	68	71	18	2	**1**
p = 10	18	5	8	12	18	24	32	35	52	61	68	71	2	**8**
p = 11	2	2	5	8	12	18	24	32	35	52	61	68	71	**11**

3.3 Specific Pass Verification & Inversions Sum: [3 Marks]

- Pass p = 3 (Key 24): Comparisons: 24 < 61 (T), 24 < 52 (T), 24 < 12 (F). Moves = 2 moves (shifts 61 and 52).
- Pass p = 6 (Key 32): Comparisons: 32 < 61 (T), 32 < 52 (T), 32 < 35 (T), 32 < 24 (F). Moves = 3 moves (shifts 61, 52, 35).
- Grand Total of Position Moves: 1 + 2 + 2 + 4 + 2 + 3 + 7 + 0 + 1 + 8 + 11 = 41 position moves.
- Total Inversions: Exactly 41 inversions. Each shift removes exactly one inversion pair.

Question 4: Selection Sort & Bubble Sort — Master Solutions (10 Marks)

Input Array: [45, 18, 72, 31, 12, 88, 25, 60] (8 elements).

4.1 Selection Sort Round-by-Round Tracing: [5 Marks]

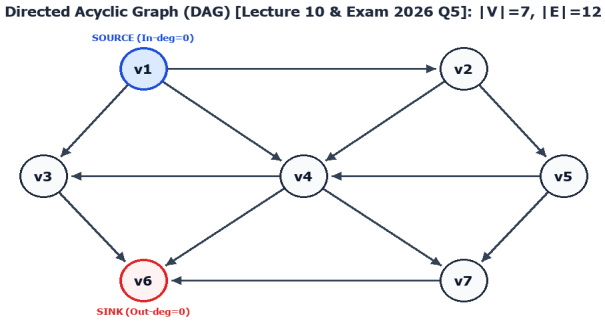
- Round 0: min_idx = 4 (val = 12) → Swap with index 0 (45) → Array: [12, 18, 72, 31, 45, 88, 25, 60]
- Round 1: min_idx = 1 (val = 18) → Already at index 1 (No swap) → Array: [12, 18, 72, 31, 45, 88, 25, 60]
- Round 2: min_idx = 6 (val = 25) → Swap with index 2 (72) → Array: [12, 18, 25, 31, 45, 88, 72, 60]
- Round 3: min_idx = 3 (val = 31) → Already at index 3 (No swap) → Array: [12, 18, 25, 31, 45, 88, 72, 60]
- Round 4: min_idx = 4 (val = 45) → Already at index 4 (No swap) → Array: [12, 18, 25, 31, 45, 88, 72, 60]
- Round 5: min_idx = 7 (val = 60) → Swap with index 5 (88) → Array: [12, 18, 25, 31, 45, 60, 72, 88]
- Round 6: min_idx = 6 (val = 72) → Already at index 6 (No swap) → Array: [12, 18, 25, 31, 45, 60, 72, 88]
- Summary: Total Passes = 7 | Comparisons = 8 × 7 / 2 = 28 comparisons | Total Swaps = 3 actual swaps (R0, R2, R5).

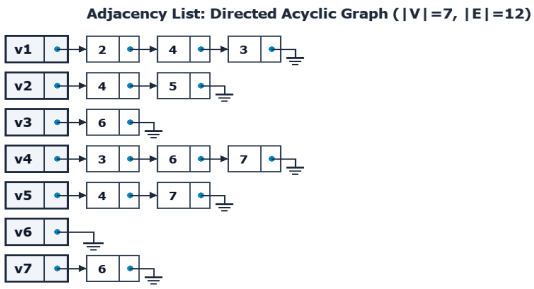
4.2 Bubble Sort Tracing & Early Termination: [5 Marks]

- Pass 1 Tracing (7 adjacent comparisons):
  (45, 18) → Swap (18, 45) | (45, 72) → OK | (72, 31) → Swap (31, 72) | (72, 12) → Swap (12, 72) | (72, 88) → OK | (88, 25) → Swap (25, 88) | (88, 60) → Swap (60, 88)
  Array after Pass 1: [18, 45, 31, 12, 72, 25, 60, 88] (Swaps in Pass 1 = 5 swaps)
- Remaining Passes:
  Pass 2: 3 swaps → [18, 31, 12, 45, 25, 60, 72, 88]
  Pass 3: 2 swaps → [18, 12, 31, 25, 45, 60, 72, 88]
  Pass 4: 2 swaps → [12, 18, 25, 31, 45, 60, 72, 88]
  Pass 5: 0 swaps → [12, 18, 25, 31, 45, 60, 72, 88] (Array is sorted, swapped = False)
- Total Swaps from start to finish: 5 + 3 + 2 + 2 = 12 swaps.
- Early Termination: Optimized Bubble Sort terminates at Pass 5 because 0 swaps occurred.

Question 5: Topological Sort (Kahn's Algorithm on 7-Vertex DAG) — Master Solutions (10 Marks)

Graph: 7 vertices, 12 directed edges. Canonical DAG from classroom whiteboard photo 840922631.





5.1 Indegree Array Processing Table across All 7 Rounds: [4 Marks]

Round	Vertex Dequeued	v_1	v_2	v_3	v_4	v_5	v_6	v_7
Initial	Start (Q: 5, 7)	2	2	2	3	**0**	3	**0**
Round 1	Dequeue 5 (Q: 7)	1	1	1	3	-	3	0
Round 2	Dequeue 7 (Q: 2)	1	**0**	1	3	-	2	-
Round 3	Dequeue 2 (Q: 1)	**0**	-	1	2	-	2	-
Round 4	Dequeue 1 (Q: 3)	-	-	**0**	1	-	2	-
Round 5	Dequeue 3 (Q: 4)	-	-	-	**0**	-	1	-
Round 6	Dequeue 4 (Q: 6)	-	-	-	-	-	**0**	-
Round 7	Dequeue 6 (Q: Empty)	-	-	-	-	-	-	-

5.2 Circular Queue Tracing (Capacity 7, Slots 0 to 6): [2 Marks]

- Enqueue Sequence: 5, 7, 2, 1, 3, 4, 6
- Dequeue Sequence: 5, 7, 2, 1, 3, 4, 6
- Final Queue Array Contents: All 7 elements were processed. In textbook implementation, array holds overwrites: [5, 7, 2, 1, 3, 4, 6] (or empty flags).
- Pointers: front = 6 | back = 6 (Wrapped back to index 6) | currentSize = 0 | Status: Empty

5.3 Final Topological Ordering: [3 Marks]

Official Correct Answer: 5, 7, 2, 1, 3, 4, 6 (or 7, 5, 2, 1, 3, 4, 6)

* CRITICAL GRADING PITFALL: Must use commas! Answers written with arrows (5 → 7 → 2...) receive 0 Marks immediately!

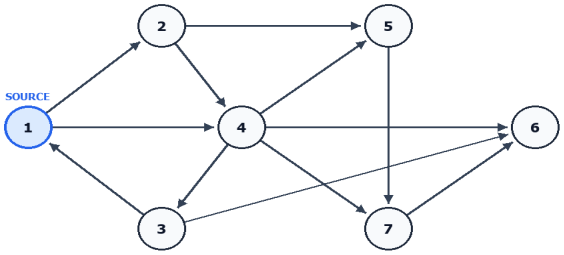
5.4 Cycle Detection & DAG Verification: [1 Mark]

- Kahn's algorithm detects cycles by tracking the count of dequeued vertices: If num_dequeued < |V| (7) when the queue becomes empty, the graph contains at least one directed cycle.
- Here, num_dequeued = 7 = |V|, proving that the graph is a valid DAG (Directed Acyclic Graph).

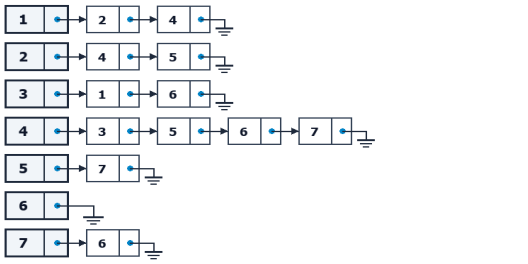
Question 6: Unweighted Shortest Path (BFS) — Master Solutions (10 Marks)

Directed Graph: 7 vertices, 12 directed edges. Canonical Shortest Path from whiteboard photo 838089388. (Source = 1).

Directed Graph G = (V, E) [Exam 2026 Question 6]: Source Vertex = 1



Adjacency List: Directed Graph (Source = 1, 7 Vertices)



6.1 BFS Execution Table for Shortest Path (Source = 1): [4 Marks]

Vertex (v)	Known (T / F)	Distance (d_v)	Previous Path (p_v)	Shortest Route from Source (1)
v_1 (Source)	T (True)	0	0 (None)	1 (Start node)
v_2	T (True)	1	1	1, 2
v_3	T (True)	2	4	1, 4, 3
v_4	T (True)	1	1	1, 4
v_5	T (True)	2	4 (or 2)	1, 4, 5 (or 1, 2, 5)
v_6	T (True)	2	4	1, 4, 6
v_7	T (True)	2	4	1, 4, 7

6.2 Circular Queue State upon Completion (Capacity 7, Slots 0 to 6): [2 Marks]

- Enqueue Sequence: 1, 2, 4, 5, 3, 6, 7 (or 1, 2, 4, 3, 5, 6, 7)
- Dequeue Sequence: 1, 2, 4, 5, 3, 6, 7
- Final Queue Contents: [1, 2, 4, 5, 3, 6, 7] (all elements processed)
- Pointers: front = 6 | back = 6 | currentSize = 0 | Status: Empty

6.3 Shortest Path Reconstruction & Distance (4 Destinations): [3 Marks]

* CRITICAL PROFESSOR'S RULE: Must write comma-separated sequences ONLY (e.g. 1, 4, 5). DO NOT write arrows (1 → 4 → 5). Distance is measured in number of EDGES!

- Shortest Path to Vertex 5: Distance = 2 edges | Route Sequence: 1, 4, 5 (or 1, 2, 5)
- Shortest Path to Vertex 7: Distance = 2 edges | Route Sequence: 1, 4, 7
- Shortest Path to Vertex 6: Distance = 2 edges | Route Sequence: 1, 4, 6
- Shortest Path to Vertex 3: Distance = 2 edges | Route Sequence: 1, 4, 3

6.4 Algorithm Complexity & BFS Property: [1 Mark]

- Time Complexity: O(|V| + |E|) when using Adjacency List (each vertex enqueued once, each edge examined once).
- BFS Correctness Property: BFS visits vertices in non-decreasing order of distance from source. In unweighted graphs, each edge has weight 1, so the first time a vertex is reached guarantees the minimum edge count. In weighted graphs, a path with more edges may have less total weight, requiring Dijkstra's algorithm.

Question 7: Advanced Theory, Structural Properties & Calculations — Master Solutions (10 Marks)

7.1 Structural Properties (Data Structures Classification): [2 Marks]

(a) Data Structure with TWO formal properties: Binary Heap (Priority Queue)
1. Structure Property: Complete Binary Tree (filled left-to-right at lowest level).
2. Heap-Order Property: Parent is smaller than or equal to children (for Min-Heap).

(b) Tree Data Structure with THREE formal properties: Binary Search Tree (BST) (or AVL Tree / B-Tree)
1. Binary Tree Structure (each node has at most 2 children).
2. Left Subtree Property: For every node X, all keys in left subtree are strictly less than key(X).
3. Right Subtree Property: For every node X, all keys in right subtree are strictly greater than key(X).

7.2 Hash Function Theoretical Properties & Realistic Limitation: [2 Marks]

• 3 Ideal Properties of a Hash Function:
1. Fast and easy to compute: O(1) constant time.
2. Uniform distribution: Distributes keys evenly across all table slots to minimize clustering.
3. Injective / Collision-free: Maps distinct keys to distinct array slots (Zero collisions).

• Impossible Property in Practice: Property 3 (Collision-free / Zero collisions)

• Two Fundamental Reasons:

Reason 1 (Pigeonhole Principle): The universe of possible keys (e.g. all strings/integers) is infinite or vastly larger than the finite hash table size (|U| >> TableSize). Collisions are mathematically inevitable.

Reason 2 (Unknown Input Distribution): In real-world applications, incoming data cannot be known in advance, causing clustering at common hash values.

7.3 Graph Memory Waste Percentage (Sparse Graph Analysis): [2 Marks]

• Total cells in Adjacency Matrix: 7 × 7 = 49 cells.
• Cells containing 1 (active edges): 12 cells → (12 / 49) × 100 = 24.49%
• Cells containing 0 (wasted space): 37 cells → (37 / 49) × 100 = 75.51% (Waste Percentage)
• Cells required in Adjacency List: |V| + |E| = 7 + 12 = 19 nodes/pointers.
• Percentage of memory saved by Adjacency List: (49 - 19) / 49 × 100 = 61.22% memory saved.

7.4 Complete Graph Edge Calculations: [1 Mark]

(a) Complete Undirected Graph K_{16} (|V| = 16):
Formula: |E| = |V|(|V| - 1) / 2 = 16 × 15 / 2 = 120 edges (Exact integer!).

(b) Complete Directed Graph with 16 vertices (no self-loops):
Formula: |E| = |V|(|V| - 1) = 16 × 15 = 240 directed edges.

7.5 Complete Binary Tree Node Bounds: [2 Marks]

Formula for height h (Root at h = 0): Minimum nodes = 2^h | Maximum nodes = 2^(h+1) - 1.
• Height h = 10: Minimum = 2^{10} = 1,024 nodes | Maximum = 2^{11} - 1 = 2,047 nodes.
• Height h = 15: Minimum = 2^{15} = 32,768 nodes | Maximum = 2^{16} - 1 = 65,535 nodes.
• Height h = 16: Minimum = 2^{16} = 65,536 nodes | Maximum = 2^{17} - 1 = 131,071 nodes.
(Answers are computed exact integers as required by grading rules)

7.6 Binary Heap 1D Array Formulas: [1 Mark]

For index i in a 1-based array:
• Left Child = 2i | Right Child = 2i + 1 | Parent = □i / 2□ (i // 2)
• Why floor division must be used: Right child at index 2i + 1 divided by 2 produces (2i + 1)/2 = i + 0.5. Truncating the decimal yields parent index i. Rounding up would yield i + 1 which references the incorrect node.

***** END OF MASTER SOLUTIONS — 100% CANONICAL CURRICULUM ALIGNED *****